



BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Compiler Design

Dr Naveen Kumar Laskari
laskari@wilp.bits-Pilani.ac.in

Compiler Design

Topic: LEX and YACC

Lex & Yacc Tutorial

What is Lex?

- The main job of a *lexical analyzer (scanner)* is to break up an input stream into more usable elements (*tokens*)

a = b + c * d ;

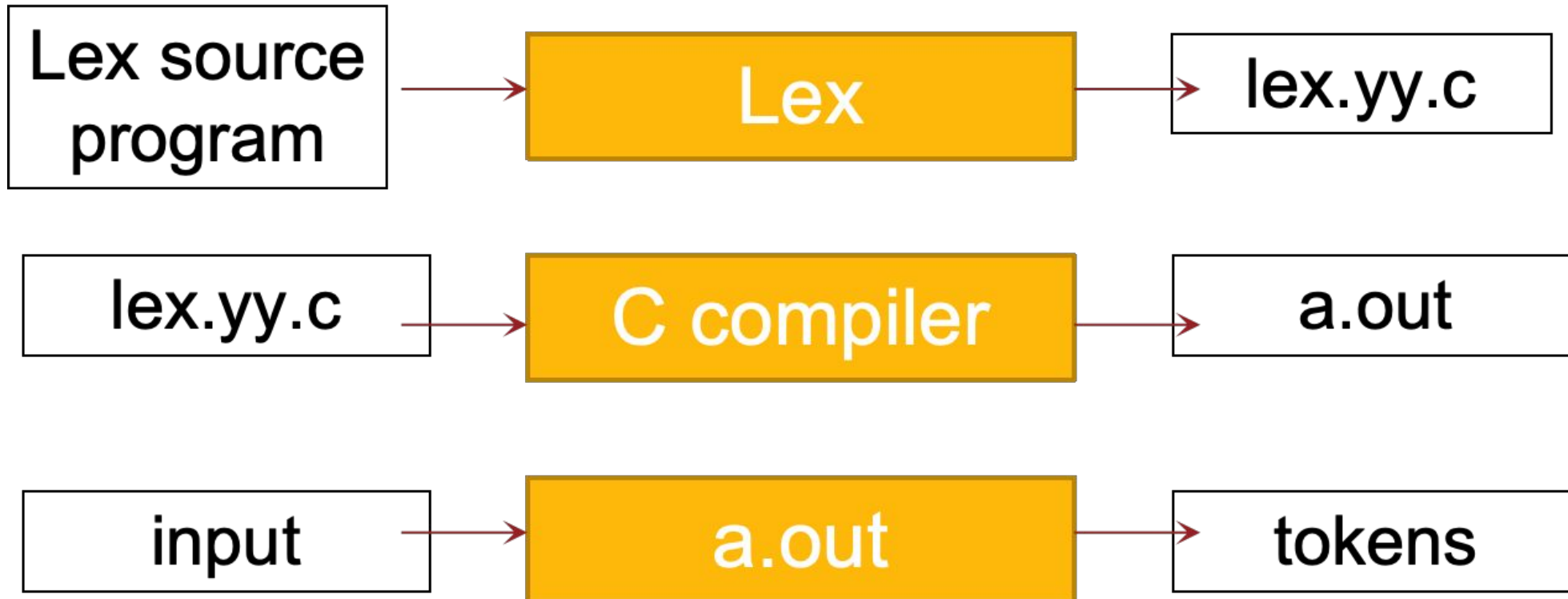
ID ASSIGN ID PLUS ID MULT ID SEMI

- Lex is an utility to help you rapidly generate your scanners

Lex – Lexical Analyzer

- Lexical analyzers **tokenize** input streams
- Tokens are the **terminals** of a language
 - English
 - words, punctuation marks, ...
 - Programming language
 - Identifiers, operators, keywords, ...
- Regular expressions define **terminals/tokens**

An Overview of Lex



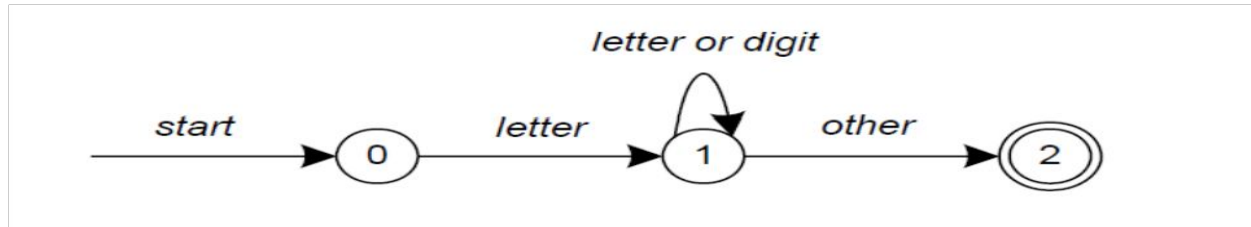
Lex

- lex is a program (generator) that generates lexical analyzers, (widely used on Unix).
- It is mostly used with Yacc parser generator.
- Written by Eric Schmidt and Mike Lesk.
- It reads the input stream (specifying the lexical analyzer) and outputs source code implementing the lexical analyzer in the C programming language.
- Lex will read patterns (regular expressions); then produces C code for a lexical analyzer that scans for identifiers.

Lex

A simple pattern: **letter(letter|digit)***

- Regular expressions are translated by lex to a computer program that mimics an FSA.
- This pattern matches a string of characters that begins with a single letter followed by zero or more letters or digits.



Lex

```
start:  goto state0

state0: read c
        if c = letter goto state1
        goto state0

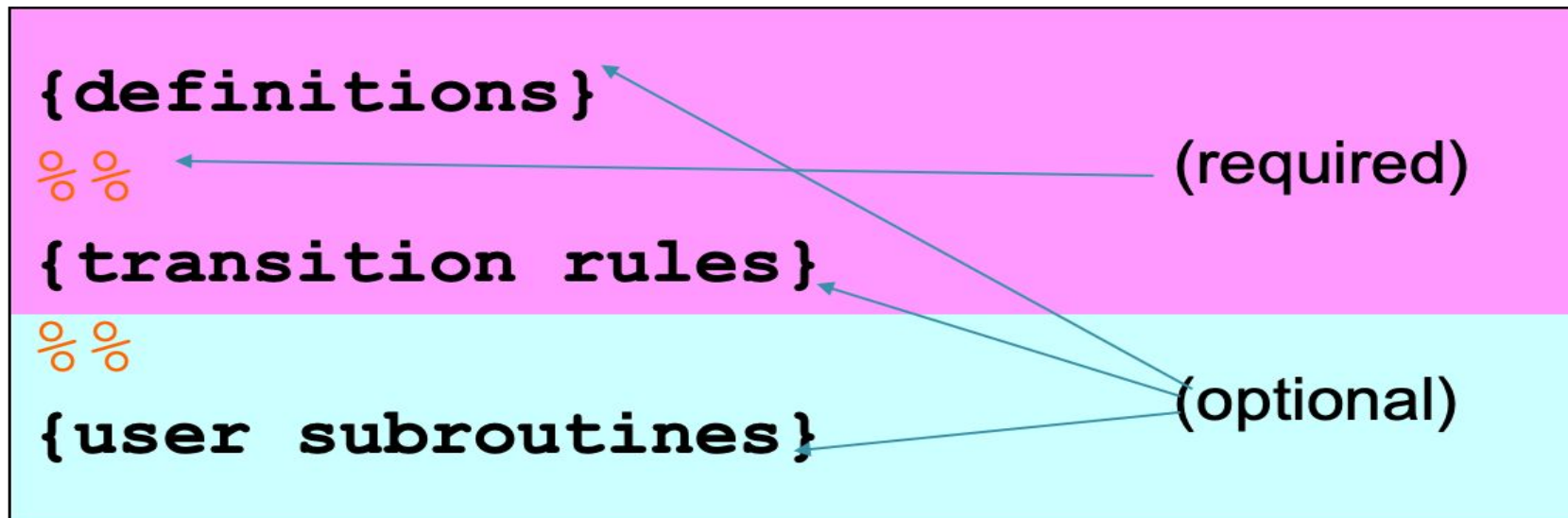
state1: read c
        if c = letter goto state1
        if c = digit goto state1
        goto state2

state2: accept string
```

- Some limitations, Lex cannot be used to recognize nested structures such as parentheses, since it only has states and transitions between states.
- So, Lex is good at pattern matching, while Yacc is for more challenging tasks.

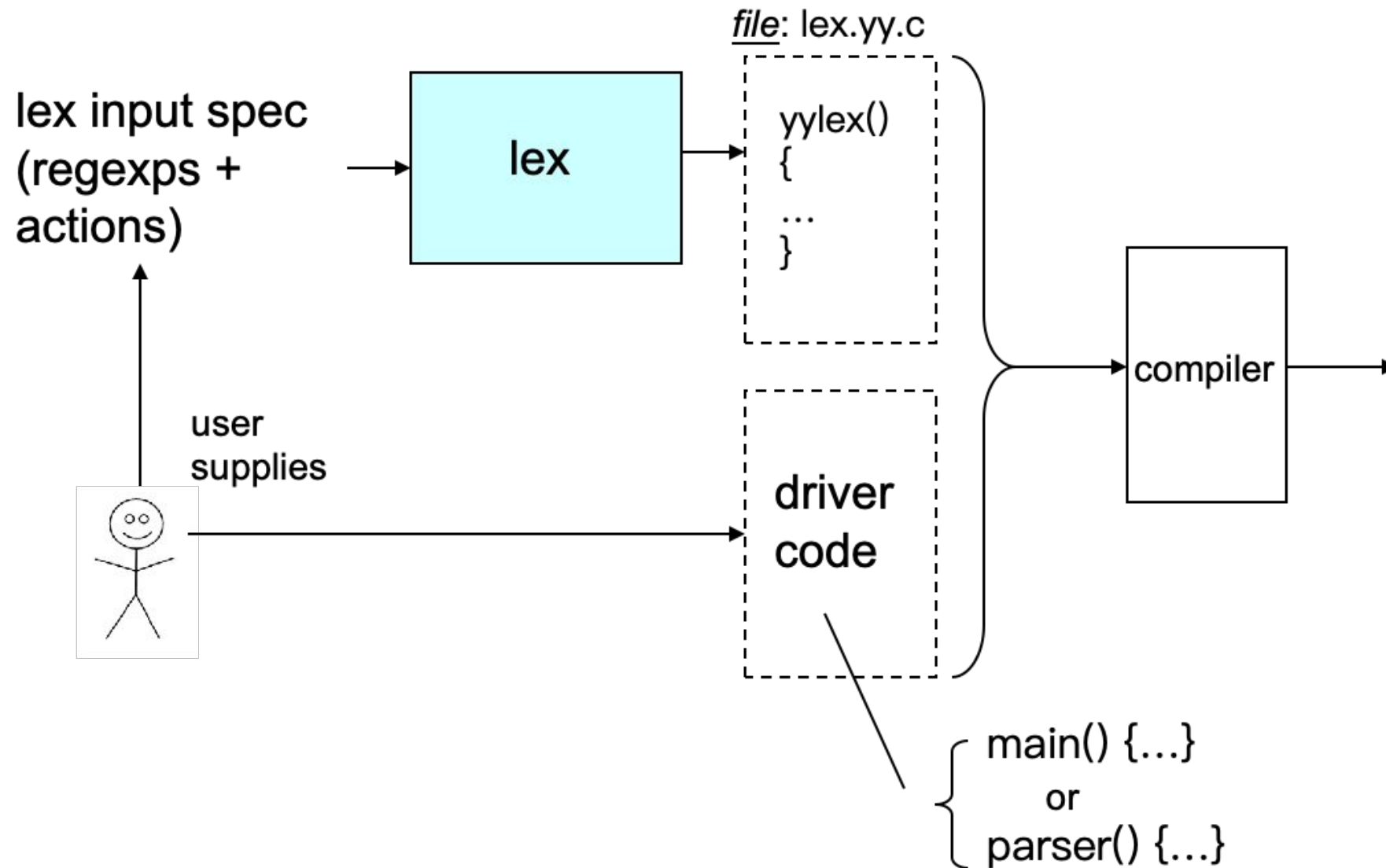
Lex Source

- Lex source is separated into **three sections** by **%%** delimiters
- The general format of Lex source is



- The absolute minimum Lex program is thus

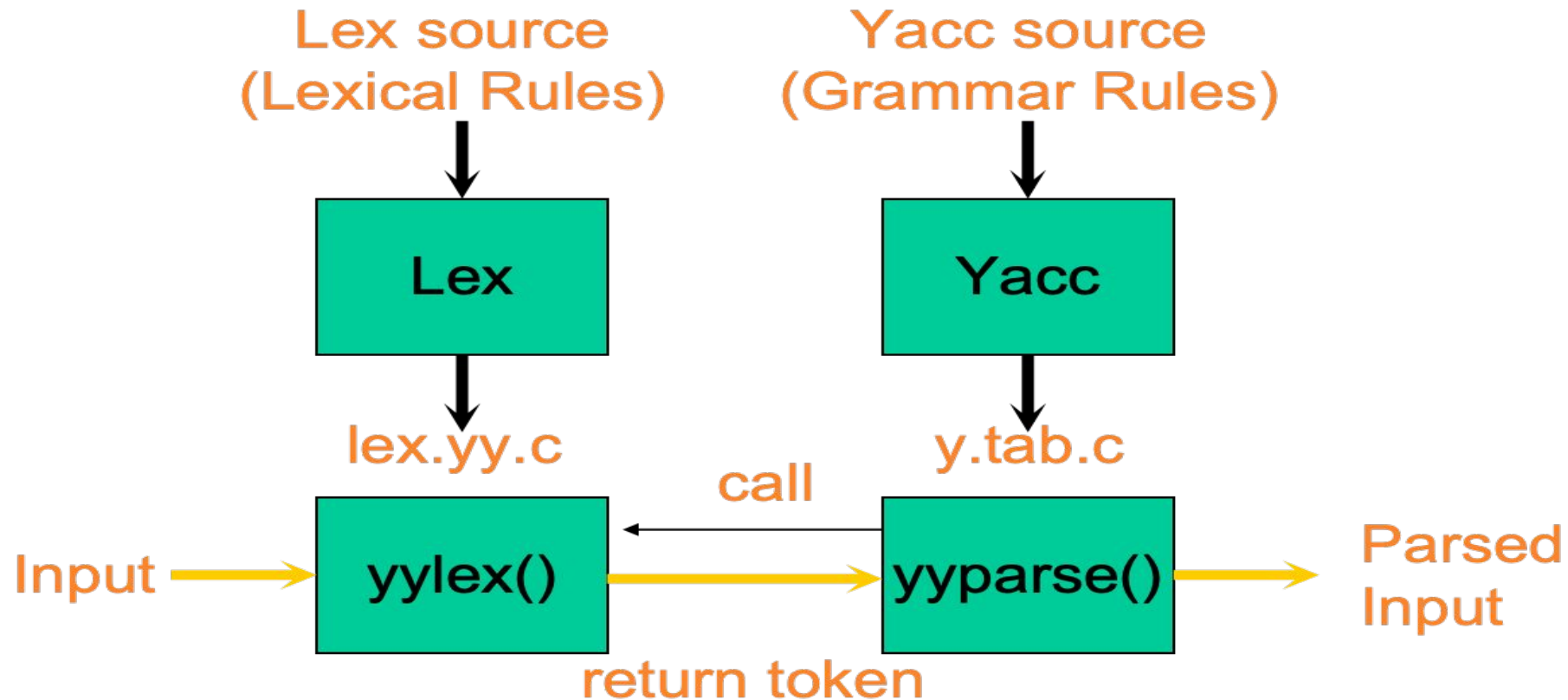
```
%%
```



Lex vs. Yacc

- Lex
 - Lex generates C code for a lexical analyzer, or **scanner**
 - Lex uses patterns that match strings in the input and converts the strings to tokens
- Yacc
 - Yacc generates C code for syntax analyzer, or **parser**.
 - Yacc uses grammar rules that allow it to analyze tokens from Lex and create a syntax tree.

Lex with Yacc



Lex Source Program

- Lex source is a table of
 - regular expressions and
 - corresponding program fragments (actions)

```
digit [0-9]
letter [a-zA-Z]
%%
{letter}({letter}|{digit})* { printf("id: %s\n", yytext); }
.|\\n                        { /* Ignore all other characters. */ }
%%
int main() {
    yylex();
    return 0;
}
```

Example

A flex program to read a file of (positive) integers and compute the average:

```
%{  
#include <stdio.h>  
#include <stdlib.h>  
%}  
dgt  [0-9]  
%%  
{dgt}+  return atoi(yytext);  
%%  
void main()  
{  
    int val, total = 0, n = 0;  
    while ( (val = yylex()) > 0 ) {  
        total += val;  
        n++;  
    }  
    if (n > 0) printf("ave = %d\n", total/n);  
}
```

A flex program to read a file of (positive) integers and compute the average:

definitions

```
%{  
#include <stdio.h>  
#include <stdlib.h>  
%}  
dgt [0-9]  
%%
```

Definition for a digit
(could have used builtin definition [:digit:] instead)

rules

```
{dgt}+ return atoi(yytext);  
%%
```

Rule to match a number and return its value to
the calling routine

user code

```
void main()  
{  
    int val, total = 0, n = 0;  
    while ( (val = yylex()) > 0 ) {  
        total += val;  
        n++;  
    }  
    if (n > 0) printf("ave = %d\n",  
        total/n);  
}
```

Driver code
(could instead have been in a separate file)

Lex Tutorial

A flex program to read a file of (positive) integers and compute the average:

definitions

```
%{  
#include <stdio.h>  
#include <stdlib.h>  
%}
```

rules

```
%%  
{dgt}+ return atoi(yytext);  
%%
```

user code

```
void main()  
{  
    int val, total = 0, n = 0;  
    while ( (val = yylex()) > 0 ) {  
        total += val;  
        n++;  
    }  
    if (n > 0) printf("ave = %d\n",  
        total/n);  
}
```

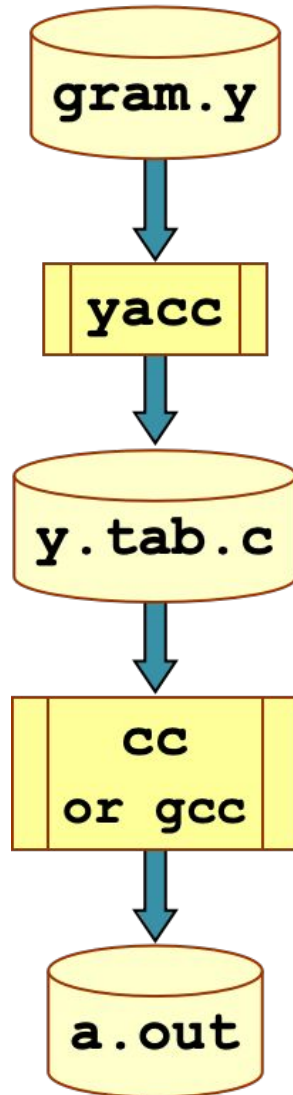
defining and using a name

What is YACC ?

Tool which will produce a parser for a given grammar.

YACC (Yet Another Compiler Compiler) is a program designed to compile a LALR(1) grammar and to produce the source code of the syntactic analyzer of the language produced by this grammar.

How YACC Works



File containing desired
grammar in yacc format

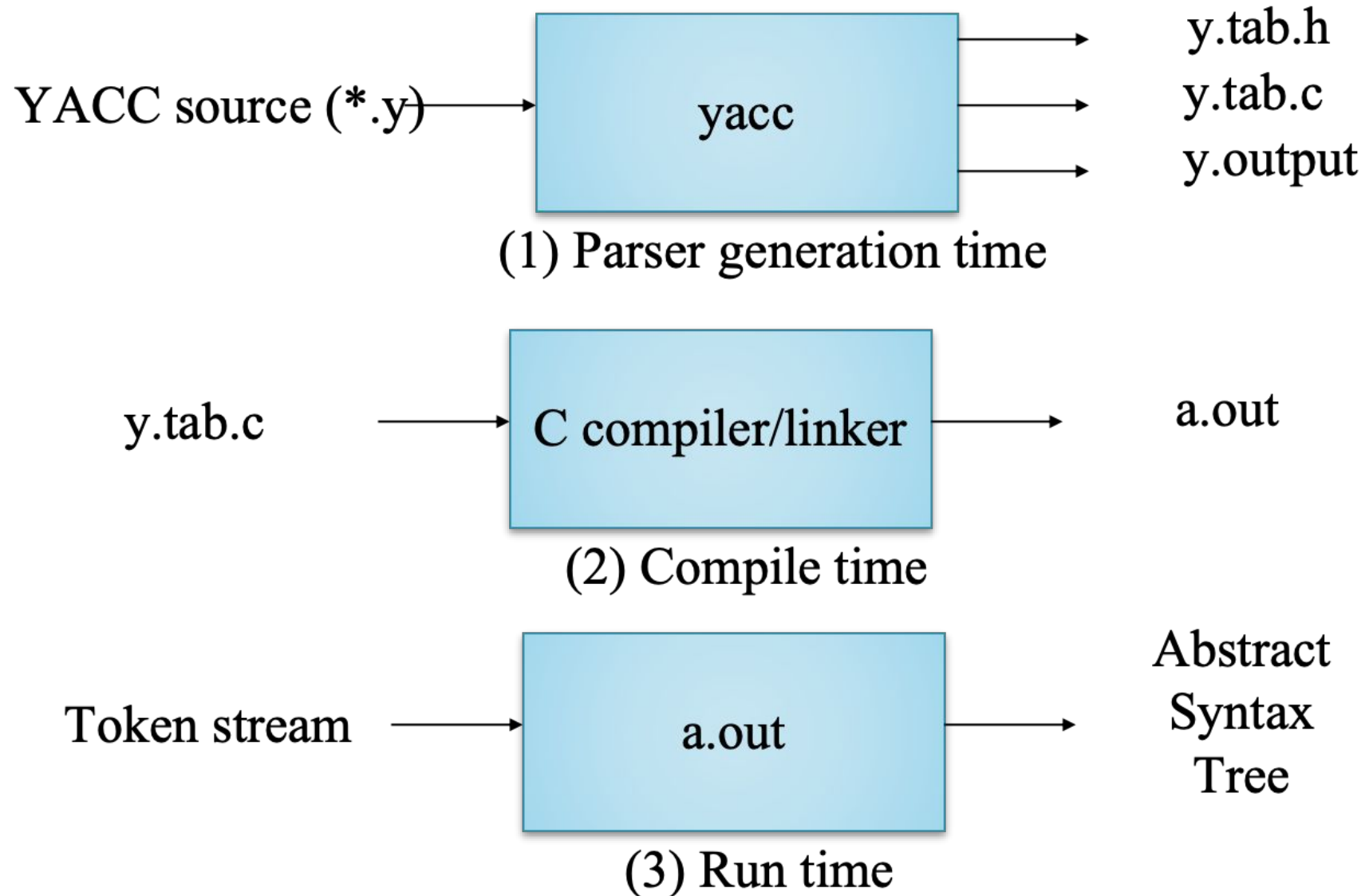
yacc program

C source program created by yacc

C compiler

Executable program that will parse
grammar given in gram.y

How YACC Works



An YACC File Example

... definitions ...

% %

... rules ...

% %

... subroutines ...

```
%{
#include <stdio.h>
%}

%token NAME NUMBER
%%

statement: NAME '=' expression
          | expression           { printf("= %d\n", $1); }
          ;

expression: expression '+' NUMBER { $$ = $1 + $3; }
           | expression '-' NUMBER { $$ = $1 - $3; }
           | NUMBER                { $$ = $1; }
           ;
%%

int yyerror(char *s)
{
    fprintf(stderr, "%s\n", s);
    return 0;
}

int main(void)
{
    yyparse();
    return 0;
}
```

YACC File Format

%{

C declarations

%}

yacc declarations

%%

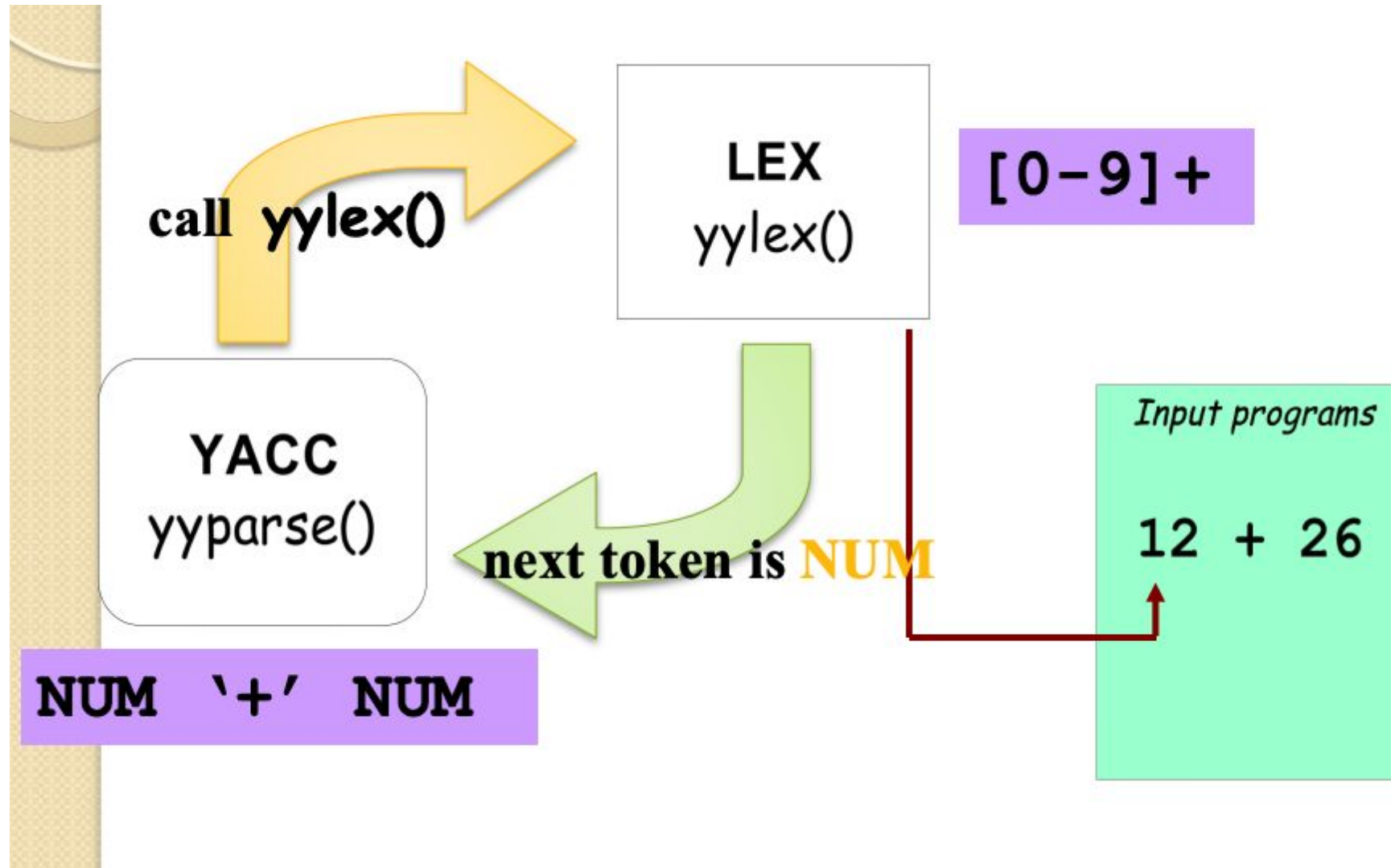
Grammar rules

%%

Additional C code

- Comments enclosed in `/* ... */` may appear in any of the sections.

Works with Lex



Yacc Example

- Taken from Lex & Yacc
- Simple calculator

a = 4 + 6

a

a=10

b = 7

c = a + b

c

c = 17

\$

expression ::= expression '+' term

|
 expression '-' term |
 term

term ::= term '*' factor |
 term '/' factor |
 factor

factor ::= '(' expression ')' |
 '-' factor |
 NUMBER |
 NAME

```

statement_list: statement '\n'
               | statement_list statement '\n'
               ;
statement: NAME '=' expression { $1->value = $3; }
          | expression          { printf("= %g\n", $1); }
          ;
expression: expression '+' term { $$ = $1 + $3; }
           | expression '-' term { $$ = $1 - $3; }
           | term
           ;
term:      term '*' factor { $$ = $1 * $3; }
        | term '/' factor { if ($3 == 0.0)
                             yyerror("divide by zero");
                             else
                                 $$ = $1 / $3;
                             }
        | factor
        ;
factor: '(' expression ')' { $$ = $2; }
       | '-' factor        { $$ = -$2; }
       | NUMBER            { $$ = $1; }
       | NAME              { $$ = $1->value; }
       ;
%%

```

parser.y

Thank You

